RockyBorg - Standalone Python + Telemetry Flow

Illustrative Python view based on the PiBorg reference and current project/src scaffold. Python vs ROS 2 remains a team decision.

ROBOT: PiBorg hardware + planned Python modules

ROCKYBORG HARDWARE

PiBorg reference:
Two rear geared DC drive motors
Front wheel steered by chassis-tilting servo
Purpose-made motor controller

Team hardware list:
Raspberry Pi 4, 8 GB
Camera Module 3 (view tilts with chassis)
Light sensor, laser distance sensor, BME280

Check actual kit revision, installed parts, and stock.

sensor / camera inputs

motor + steering commands

project/src - STANDALONE PYTHON OPTION

main.py - planned entry / orchestration
config.py - planned settings
camera/camera.py - capture frames
camera/line_detection.py - estimate line position
sensors/light_sensor.py
sensors/distance_sensor.py
sensors/environment_sensor.py
navigation/line_follower.py
navigation/controller.py
navigation/obstacle_avoidance.py
hardware/motors.py - drive + tilt-steer commands via the PiBorg controller (API TBD)
communication/mqtt_client.py - candidate telemetry publisher

All .py files are empty placeholders. Roles above are inferred from filenames and project notes.

selected numeric telemetry

TELEMETRY: message flow, storage, dashboard

MQTT broker

Additional service
Candidate transport

Broker / host: TBD

Node-RED flow

Receives, prepares, and displays telemetry

MQTT In
Subscribe to topic

JSON
Decode payload

Debug
Inspect during setup
message topic

Switch / Change
Validate, route, shape fields

live display

FlowFuse Dashboard
Node-RED Dashboard 2.0
Live telemetry UI
@flowfuse/node-red-dashboard

optional history

InfluxDB Out (optional)
Install compatible palette package

write timestamped points

InfluxDB (optional)

Time-series storage
Timestamp + run ID + values

Version / bucket: TBD

COLOR + LINE LEGEND

- PiBorg kit / Pi / listed sensors
- Python src files - empty scaffold
- MQTT transport - candidate
- Node-RED flow nodes
- InfluxDB time-series storage
- FlowFuse Dashboard (Node-RED Dashboard 2.0)
- Dashed outline - TBD / unverified

Solid dark arrow: sensor input or local motor/control flow

Dashed arrows: selected telemetry sent toward storage

Purple arrow: Node-RED sends live telemetry to FlowFuse Dashboard

HOW TO READ
1. Camera and sensor readings enter Python modules.
2. Navigation code chooses movement; hardware/motors.py sends drive and tilt-steering commands through the PiBorg controller.
3. Selected numeric values branch to MQTT and Node-RED, which sends live values to FlowFuse Dashboard and can optionally store time-series data in InfluxDB.

REFERENCE + STATUS
PiBorg reference describes two rear geared motors, a front wheel, and servo-tilt steering; the team order lists the Pi, camera, and sensors. Confirm exact kit revision and installed parts.
MQTT, service locations, database version, sensor interfaces, and motor-control API are open. FlowFuse Dashboard is a candidate palette package; InfluxDB storage is optional. Camera frames stay out of the telemetry database. This drawing is proposed, not implemented.